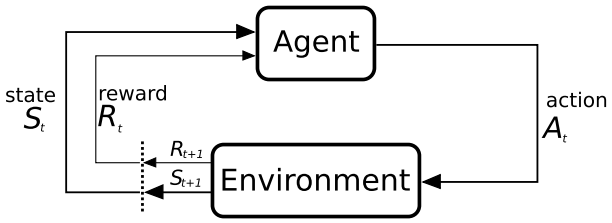
Reinforcement Learning

TAMU ECEN-743

Brody Jordan

Spring 2026



Acknowledgements

These notes were compiled from the course materials for ECEN-743 (Reinforcement Learning) taught by Dr. Dileep Kalathil at Texas A&M University during the Spring 2026 semester.

Contents

Chapter 1

Introduction

Jan. 12

We define machine learning as a set of methods that can automatically detect patterns in data, and then use the uncovered patterns to predict future data, or to perform other... - Murphy

Supervised learning maps an input to an output based on example input-output pairs (labelled data)

Examples: image classification, email spam filtering, speech recognition

Unsupervised learning is the task of inferring a function that describes the structure of "unlabeled" data

Examples: clustering, dimensionality reduction (e.g. PCA, autoencoder)

Reinforcement learning is an area of ML concerned with how intelligent agents ought to take actions in an environment in order to maximize the notion of cumulative reward

Reinforcement learning challenges: no prior data (RL agent generates data, by taking action in an unknown environment), data generated is not i.i.d, no supervisor (optimal policy has to be learnt from the rewards observed in the self-generated data), actions have long term consequences (rewards are often delayed), dynamical systems issues (feedback and stability)

AlphaGo

Chapter 2

Markov Decision Process (MDP): Introduction

Chapter 3

Markov Decision Process (MDP)

Jan. 28

3.1 Q-value function

Value function of policy π is defined as

$$V_\pi(s) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) | s_0 = s]$$

Action-value function for policy π is defined as

$$Q_\pi(s, a) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) | s_0 = s, a_0 = a]$$

It is easy to show

$$Q_\pi(s, a) = r(s, a) + \gamma \sum_{s'} P(s'|s, a) V_\pi(s')$$

this is the instantaneous reward for (s, a) and the expected value of the next state.

3.2 Value Policy Iteration

Fixed point

Let $f : \mathcal{U} \to \mathcal{U}$. u^* is a fixed point of $f(\cdot)$ if $f(u^*) = u^*$.

Let $f : \mathcal{U} \to \mathcal{U}$ and let $\|\cdot\|$ be a norm defined on $\mathcal{U}$. We say $f(\cdot)$ is a contraction mapping (or simply, contraction) if $\|f(u) - f(v)\| \leq \alpha \|u - v\|$ for an $\alpha \in (0, 1)$ and for all $u, v \in \mathcal{U}$.

Banach Fixed Point Theorem.

$$\|u_{n+1} - u_n\| \leq \alpha^n \|u_1 - u_0\|$$

converges when $\alpha \leq 1$. We can show that the sequence $\{u_0, u_1, u_2, \dots\}$ is a Cauchy sequence. Since $\mathcal{U}$ is closed and bounded, the sequence converges, i.e., there exists a limit $u^* = \lim_{k\to\infty} u_k$. Since $f(\cdot)$ is a contraction, it is also a continuous function. Then, $u^* = \lim_{k\to\infty} u_k = f(\lim_{k\to\infty} u_k) = f(u^*)$.

Proof of uniqueness: suppose there exists another fixed point $\bar{u}^* \neq u^*$. Then,

$$\|u^* - \bar{u}^*\| = \|f(u^*) - f(\bar{u}^*)\| \leq \alpha \|u^* - \bar{u}^*\| .$$

This is a contradiction, so $u^* = \bar{u}^*$.

3.3 Bellman Optimality Equation

Let V^* be the optimal value function. Then, V^* satisfies the equation

$$V^*(s) = \max_a \left(r(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right)$$

Suppose the current state is s and we take action a. From the next state onwards, we follow the optimal policy π^*. What is the 'value' of state s (expected cumulative discounted reward) under this procedure?

$$V(s) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V^*(s')]$$

reward for (s,a) plus *gamma* times the expected value of the next state under the optimal policy π^*. The best action we can take in the current state s is the action which maximizes the value $V(s)$. The 'value' of state s if we take this action is

$$\max_a (r(s,a) + \gamma \mathbb{E}_{s \sim P(\cdot|s,a)}[V^*(s')])$$

3.4 Value Iteration

Bellman Operator $T : \mathbb{R}^{|\mathcal{S}|} \to \mathbb{R}^{|\mathcal{S}|}$ is defined as

$$(TV)(s) = \max_a (r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V(s')])$$

Aim to design feedback, with $V_{k+1} = TV_k$, for which we want to prove converge.

Proposition: Bellman operator T is a contraction with respect to $\|\cdot\|_\infty$, i.e., for any $V_1, V_2 \in \mathcal{R}^{|S|}$, $\|TV_1 - TV_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty$.

A useful result for the proof. Let g, h be any two real-valued function. Then,

$$|\max_a g(x,a) - \max_a h(x,a)| \leq \max_a |g(x,a) - h(x,a)|$$

Proof (of contraction property)

$$\begin{aligned}
|(TV_1)(s) - (TV_2)(s)| &= |\max_{a \in \mathcal{A}} (r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V_1(s')]) - \max_{a \in \mathcal{A}} (r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V_2(s')])| \\
&\leq \max_{a \in \mathcal{A}} |(r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V_1(s')]) - (r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V_2(s')])| \\
&= \max_{a \in \mathcal{A}} |\gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V_1(s')] - \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V_2(s')])| \\
&\leq \max_{a \in \mathcal{A}} \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[|V_1(s') - V_2(s')|] \\
&\leq \max_{a \in \mathcal{A}} \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[\|V_1 - V_2\|_\infty] = \gamma \|V_1 - V_2\|_\infty
\end{aligned}$$

this implies

$$\|TV_1 - TV_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty$$

Chapter 4

Value Iteration and Policy Iteration

Feb. 2

Bellman Optimality Equation

$$V^*(s) = \max_a \left(r(s,a) + \gamma \sum_{s'} P(s'|s,a) V^*(s') \right)$$

The statement that this is a fixed-point has nothing to do with contraction. We want to perform an iterative loop using the Bellman operator. The optimal value function V^* is the unique fixed point of the Bellman operator T, i.e., $TV^* = V^*$. Value iteration, defined as $V_{k+1} = TV_k$, with an arbitrary initial value V_0 will converge to the optimal value function V^*, i.e. $\lim_{k\to\infty} V_k = V^*$.

Optimal policy π^* can be computed as

$$\pi^*(s) = \arg\max_a (r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V^*(s')])$$

Optimal Q-value function of a policy π

$$Q_\pi(s,a) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) | s_0 = s, a_0 = 0]$$

Just like optimal value function, you can define an optimal Q-value function

$$Q^*(s,a) = \max_\pi Q_\pi(s,a)$$

There exists a conversion from V^* to Q^* through

$$Q^*(s,a) = r(s,a) = \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V^*(s')]$$

And from Q^* to V^*

$$V^*(s) = \max_a Q^*(s,a)$$

Also from π^* to Q^*

$$\pi^*(s) = \arg\max_a Q^*(s,a)$$

In some problems, you may not know the transition probability.

Optimal Q-value function Q^* satisfies the Bellman equation

$$Q^*(s,a) = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[\max_b Q^*(s',b)]$$

Define the mapping $F : \mathbb{R}^{|\mathcal{S}||\mathcal{A}|} \to \mathbb{R}^{|\mathcal{S}||\mathcal{A}|}$ as

$$(FQ)(s,a) = r(s,a) = \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[\max_b Q(s',b)]$$

4.1 Policy Iteration

Recap of policy iteration: at every point, we evaluate how good our situation is (policy, π_k). Our value function V_π satisfies the equation $V_\pi = R_\pi + \gamma P_\pi V_\pi$. How do we find V_π?

Can we find the optimal policy by 'cleverly' searching over the policy space?

Policy iteration: for $k = 0, 1, 2, \ldots$ Policy evaluation: compute the value of the policy to get V_{π_k}

$$\text{Solve } T_{\pi_k} V_{\pi_k} = V_{\pi_k}$$

Policy improvement: perform greedy update to get the next policy π_{k+1}:

$$\pi_{k+1}(x) = \arg\max_a (r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)}[V_{\pi_k}(s')])$$

Proposition (convergence of policy iteration): $V_{\pi_{k+a}} \geq V_{\pi_k}$ and $\left\| V_{\pi_{k+1}} - V^* \right\|_\infty \leq \gamma \left\| V_{\pi_k} - V^* \right\|_\infty$.

Policy iteration: Lemma (monotone property of T_π): let $V_1 \geq V_2$. Then, $T_\pi V_1 \geq T_\pi V_2$.

Proof: note that $V_{\pi_k} = T_{\pi_k} V_{\pi_k} \leq T V_{\pi_k} = T_{\pi_{k+1}} V_{\pi_k}$

Using the monotone property of T_π,

$$V_{\pi_k} \leq T_{\pi_{k+1}} V_{\pi_k} \leq T^{(1)}_{\pi_{k+1}} V_{\pi_k} \leq \cdots \leq T^{(2)}_{\pi_{k+1}} \ldots$$

Chapter 5

TD Learning and Q-learning

Chapter 6

RL with Function Approximation

Feb. 16

6.1 Linear function approximation

Feature vector:

$$\text{For each } s \in \mathcal{S}, \phi(s) \in \mathbb{R}^d, \phi(s) = (\phi_1(s), \phi_2(s), \ldots, \phi_d(s))^T$$

Value function is approximated as, $V(s; w) = w^T \phi(s)$, where $w \in \mathbb{R}^d$ is the weight vector. Typically, $d \ll |\mathcal{S}|$. Value function approximation is linear in w.

6.2 Approximation as projection

How do we find the best approximation for $V \in \mathcal{V}$ in the set $\mathcal{V}_\phi = \{\Phi w, w \in \mathbb{R}^d\}$

$$V_{\text{approx}} = \arg\min_{\tilde{V} \in \mathcal{V}_\phi} \left\| V - \tilde{V} \right\|$$

6.3 Weighted norm and contraction

T_π is a contraction w.r.t. $\|\cdot\|_\infty$, but not w.r.t. $\|\cdot\|_2$. Π is non-expansive w.r.t. $\|\cdot\|_2$, but expansive w.r.t. $\|\cdot\|_\infty$. How do we ensure that the iteration $V_{k+1} = \Pi T_\pi V_k$ will converge? Define the weighted L^2 norm, $\|V\|_{2,\mu} = \mathbb{E}_{s\sim\mu}[V^2(s)]$ where μ is a probability distribution over s.

6.3.1 Jensen's Inequality

Let $f : \mathbb{R}^n \to \mathbb{R}$ be a convex function and let X be a random vector. Then, $f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$.

Lemma 6.3.1. Contraction property of T_π w.r.t. $\|\cdot\|_{2,\mu}$. Let μ be the stationary distribution corresponding to P_π, i.e., $\mu P_\pi = \mu$. Then, $\|T_\pi V_1 - T_\pi V_2\|_{2,\mu} \leq \gamma \|V_1 - V_2\|_{2,\mu}$.

Proof. We will first show the following: let μ be the stationary distribution corresponding to P_π,

i.e., $\mu P_\pi = \mu$. Then, $\|P_\pi V\|_{2,\mu} \leq \|V\|_{2\mu}$.

$$\begin{aligned}\|P_\pi V\|_{2,\mu}^2 &= \mathbb{E}_{s\sim\mu}[((P_\pi V)(s))^2] = \sum_{s\in\mathcal{S}} \mu(s) \left(\sum_{s'\in\mathcal{S}} P_\pi(s,s')V(s')\right)^2 \\ &\leq \sum_{s\in\mathcal{S}} \mu(s) \sum_{s'\in\mathcal{S}} P_\pi(s,s')V^2(s') \\ &= \sum_{s'\in\mathcal{S}} \sum_{s\in\mathcal{S}} \mu(s)P_\pi(s,s')V^2(s') \\ &= \sum_{s'\in\mathcal{S}} \mu(s')V^2(s') = \|V\|_{2,\mu}^2\end{aligned}$$

Then,

$$\|T_\pi V_1 - T_\pi V_2\|_{2,\mu} = \|(r_\pi)\| \ldots$$

□

6.3.2 Approximation error

ΠT_π is a contraction mapping. So, it has a unique fixed point $\bar{V}_\pi$. The iteration $V_{k+1} = \Pi T_\pi V_k$ will converge to $\bar{V}_\pi$. What is the approximation error $\left\|V_\pi - \bar{V}_\pi\right\|_{2,\mu}$?

Proposition (approximation error)

$$\|\Pi V_\pi - V_\pi\|_{2,\mu} \leq \left\|\bar{V}_\pi - V_\pi\right\|_{2,\mu} \leq \frac{1}{\sqrt{1-\gamma^2}} \|\Pi V_\pi - V_\pi\|_{2,\mu}$$

6.4 TD Learning with Function Approximation

Chapter 7

Deep Q-Learning

Feb. 23

Similar to homework questions. Some proof, think-through questions. Mix of intuition, understanding, and some mathematical details. Typically one challenging question at the end of the exam. Up to and including Deep Q-Learning.

7.1 Experience Relay

Put all data in a buffer and randomly sample.

7.2 Target Network

Q-learning gradient update:

$$w = w + \alpha(r_k + \gamma \max_b Q(w)(s_{k+1}, b) - Q(w)(s_k, a_k))\nabla_w Q(w)(s_k, a_k)$$

In supervised learning, the target does not depend on the parameter. In Q-learning, target $r_k + \gamma \max_b Q(w)(s_{k+1}, b)$ depends on w, so the target is moving as the parameter is updated.

7.3 DQN Algorithm

7.4 Maximization Bias in Q-Learning

Let $\hat{\pi} = \arg\max_a \hat{Q}(s, a)$ and $\hat{V}_{\hat{\pi}} = \max_a \hat{Q}(s, a)$.

$$\mathbb{E}[\hat{V}_{\hat{\pi}}] = \mathbb{E}[\max_a \hat{Q}(s, a)] \geq \max_a \mathbb{E}[\hat{Q}(s, a)]$$

The greedy policy obtained from the estimated Q values can yield a maximization bias when the number of samples are finite. Maximization bias is DQN, from [Van Hasselt et al., 2016].

7.5 Double Estimator

Consider two random variables Y^1 and Y^2 with mean values μ^1 and μ^2. Let $\mathcal{Y}^1$ and $\mathcal{Y}^2$ be the sets of i.i.d. samples of Y^1 and Y^2. Our goal is to esimate $\max_i \mu^i$. With a single estimator, we let $\hat{\mu}^i = \frac{1}{|\mathcal{Y}^i|} \sum_{y \in \mathcal{Y}^i} y$ and let $\hat{i}^* = \arg\max_i \hat{\mu}^i$. Then, single esitmator is $\hat{\mu}_{\max} = \hat{\mu}^{\hat{i}^*}$. This single estimator gives an overestimate because we are using the same samples both to determine

the maximizer and value.

Possible solution is to divide the samples into two sets and use them to learn two independent estmates. Now we split $\mathcal{Y}^i$ into $\mathcal{Y}_A^i$ and $\mathcal{Y}_B^i$ and then compute $\hat{\mu}_A^i = \frac{1}{|\mathcal{Y}_A^i|}\sum_{y\in\mathcal{Y}_A^i} y$ and $\hat{\mu}_B^i = \frac{1}{|\mathcal{Y}_B^i|}\sum_{y\in\mathcal{Y}_B^i} y$. Now find $\hat{i}^* = \arg\max_i \hat{\mu}_A$ and get double estimator $\hat{\mu}_{\max} = \hat{\mu}_B^{\hat{i}^*}$. We can show that the double estimator is an underestimator, i.e., $\mathbb{E}[\hat{\mu}_{\max}] \leq \max_i \mathbb{E}[Y^i]$.

Proof. Let $i^* = \arg\max_i \mathbb{E}[Y^i]$ be the true maximizer. Then,

$$\begin{aligned}
\mathbb{E}[\hat{\mu}_{\max}] = \mathbb{E}[\hat{\mu}_B^{\hat{i}^*}] &= \mathbb{P}(\hat{i}^* = i^*)\mathbb{E}[\hat{\mu}_B^{\hat{i}^*} \mid \hat{i}^* = i^*] + \mathbb{P}(\hat{i}^* \neq i^*)\mathbb{E}[\hat{\mu}_B^{\hat{i}^*} \mid \hat{i}^* \neq i^*] \\
&= \mathbb{P}(\hat{i}^* = i^*) \max_i \mathbb{E}[Y^i] + \mathbb{P}(\hat{i}^* \neq i^*)\mathbb{E}[\hat{\mu}_B^{\hat{i}^*} \mid \hat{i}^* \neq i^*] \\
&\leq \mathbb{P}(\hat{i}^* = i^*) \max_i \mathbb{E}[Y^i] + \mathbb{P}(\hat{i}^* \neq i^*) \max_i \mathbb{E}[Y^i] = \max_i \mathbb{E}[Y^i]
\end{aligned}$$

□

7.6 Double DQN

Use one Q-network to select the max-action and another Q-network to evaluate it. DQN maintains two networks anyway (one as the target network and one as the current network)

$$w = w + \alpha(R + \gamma \max_b Q(w^-)(s', b) - Q(w)(s, a))\nabla Q(w)(s, a)$$

where $w^- = w$, after every N steps. To implment double DQN, use the current Q-network to select the max-action, and the target Q-network to evaluate that action

$$w = w + \alpha(r + \gamma Q(w^-)(s', \arg\max_b Q(w)(s', b)) - Q(w)(s, a))\nabla Q(w)(s, a)$$

7.6.1 Advantage Function

The advantage function is defined as

$$A_\pi(x, a) = Q_\pi(x, a) - V_\pi(x)$$

The value function measures how good it is to be in a particular state; the Q function measures the value of choosing particular action when in that state; the advantage function obtains a relative measure of the importance of each action.

7.6.2 Dueling DQN Architecture

In DQN, there is a single stream to estimate Q function. In DDQN, two streams separately estimate (scalar) state-value and the advantages for each action. These two streams are particularly useful in states where its actions do not affect the environment in any relevant way. Dueling architecture can more quickly idenitify the correct action during policy evaluation as redundant or similar actions are added to the learning problem.

7.6.3 Prioritized Experience Replay (sampling from Replay Buffer)

Some experience (data samples (s_i, a_i, r_i, s_{i+1})) may be more informative than other. In experience relay, samples are taken uniformly at random from the replay buffer. Uniform sampling will not be able to make use of such informative samples efficiently.

Can we prioritize the samples to accelerate the learning progress? Intuition: prioritize a sample based on how much we can learn from a transition (expected learning progress). Proxy for this is TD error: for $e_i = (s_i, a_i, r_i, s_{i+1})$, TD error is defined as

$$\delta_i = r_i + \gamma \max_b Q(w)(s_{i+1}, b) - Q(w)(s_i, a_i)$$

In prioritized experience replay, sample e_i is selected with probability p_i,

- Proportional sampling: $p_i = \frac{|\delta_i|^c}{\sum_i |\delta_i|^c}$
- Rank-based sampling: $p_i = \frac{1}{\text{rank}(e_i)}$
  where the rank of e_i when the repla memory is sorted according to $|\delta_i|$.

Priorities experience replay offers a performance improvement over DQN.

7.6.4 Rainbow DQN

The deep reinformcent learning community has made several independent improvements to the DQN algorithm. However, it is unclear which of these extensions are complementary and can be fruitfully combined. Hessel et al., 2018, shows that the combination provides state-of-the-art performance on the Atari 2600 benchmark, both in terms of data efficiency and final performance.

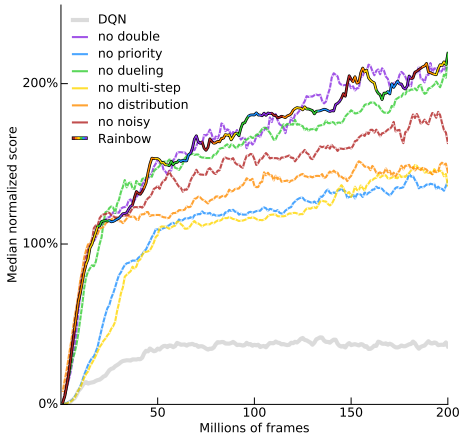


Figure 7.1: Median human-normalized performance

Later on, Agent57 was the first deep reinformcent learning agent to obtain a score that is above the human baseline on all 57 Atari 2600 games.

Chapter 8

Policy Gradient Theorem

Chapter 9

Natural Policy Gradient

Chapter 10

Trust Region Methods

Chapter 11

Advanced Actor-Critic Algorithms

11.1 Soft Actor Critic (SAC) Algorithm

11.1.1 Entropy Regularized RL

Standard RL objective:

$$\max_{\pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)], \quad \text{where, } a_t \sim \pi(s_t, \cdot)$$

Entropy regularized RL objective:

$$\max_{\pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) - \alpha \log \pi(s_t, a_t))], \quad \text{where, } a_t \sim \pi(s_t, \cdot)$$

Entropy of a distribution p:

$$H(p) = -\sum_{x} p(x) \log p(x)$$

So, this is also called maximum entropy RL:

$$\max_{\pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \alpha H(\pi(s_t, \cdot)))], \quad \text{where, } a_t \sim \pi(s_t, \cdot)$$

Why max-entropy?

You are essentially trying to find a policy that maximized the cumulative reward. What it also says is that you want to maximize the cumulative reward, but select a stochastic policy with a little more randomness to it. We want to introduce some randomness into action choice.

It gives nicer objective functions from an optimization perspective.

11.2 Soft Values

Standard value function

$$V_{\pi,\mu} = \mathbb{E}_{s\sim\mu}[V_\pi(s)]$$

Soft value function:

$$V^s_{\pi,\mu} = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) - \alpha \log(\pi(s_t, \cdot)))], \quad \text{where, } a_t \sim \pi(s_t, \cdot)$$

Denoting $H(\pi, \mu) = \mathbb{E}_{s\sim\rho_{\pi,\mu}}[H(\pi(s, \cdot))]$, we get

$$V^s_{\pi,\mu} = V_{\pi,\mu} + \alpha H(\pi, \mu)$$

Denoting π^* and π^{*s} as the optimal policies of standard and regularized objectives, we get

$$V_{\pi^*,\mu} \leq V^s_{\pi^{*s},\mu} \leq V_{\pi^*,\mu} + \frac{\alpha \log |\mathcal{A}|}{(1-\gamma)}$$

So, π^{*s} could also be nearly optimal in terms of the standard value function, as long as the regularization parameter α is chosen to be sufficiently small.

11.2.1 Form of the Optimal Soft Policy

We want to solve:

$$\max_{\pi} \mathbb{E}_{a\sim\pi(s,\cdot)}[Q(s, a) - \alpha \log \pi(s, a)]$$

Note that state s is fixed. Writing it as $\sum_a \pi(s, a)Q(s, a) - \alpha \sum_a \pi(s, a) \log \pi(s, a)$, and taking gradient w.r.t. π and equating it to zero, we get:

Optimal soft policy has the softmax form:

$$\pi^{*s}(s, a) \propto \exp(Q(s, a)/\alpha)$$

More precisely,

$$\pi^{*s}(s, a) = \exp\left(\frac{1}{\alpha}(Q^{*s}(s, a) - V^{*s}(s))\right), \text{ and,}$$
$$V^{*s}(s) = \alpha \log\left(\sum_a \exp(Q^{*s}(s, a)/\alpha)\right)$$

11.2.2 Soft Policy Evaluation

$$T^s_\pi Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s'\sim P(s,a)}[V(s')], \text{ where, } V(s) = \mathbb{E}_{a\sim\pi(s,\cdot)}[Q(s, a) - \alpha \log \pi(s, a)]$$

11.2.3 Soft Bellman Operator

11.2.4 Soft Policy Iteration

Similar to the standard policy iteration. Start with an initial policy π_0. For each k, perform soft policy evaluation and soft policy update.

Soft policy evaluation: Given the policy π_k, compute $Q^s_{\pi_k}$

Soft policy update: Given Q^s_π, update the policy to get $\pi_{k+1}(s, a) = \frac{1}{Z_{\pi_k}(s)} \exp(Q^s_{\pi_k}(s, a)/\alpha)$.

11.3 Off-policy Learning

Policy optimization algorithms such as PG, TRPO, PPO are on-policy learning algorithms. On-policy algorithms require new samples to be collected for every update of the policy; data collected from the previous iterates is discarded in on-policy algorithms. On-policy data collection is expensive (in terms of the number of gradient steps and samples per step needed); this is exacerbated in high dimensional complex tasks. Off-policy learning algorithms can reuse the data from all learning episodes (recall the replay buffer in DQN).

SAC is an off-policy policy optimization algorithm.

11.4 Soft Actor-Critic Algorithm

11.4.1 Q-Value update

Function approximation for soft Q-function Q_θ and policy π_ϕ. Instead of running evaluation for each policy π_k and performing improvement using $Q^s_{\pi_k}$, SAC alternate between optimizating θ, ϕ networks with stochastic gradient descent.

Soft Q-function parameter update: recall that $V^s_\pi(s) = \mathbb{E}_{a\sim\pi(s,\cdot)}[Q^s_\pi(s,a) - \alpha \log \pi(S,a)]$. So, we define the loss function $L_Q(\theta)$ as

$$L_Q(\theta) = (\frac{1}{2})\mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}[\Big(Q_\theta(s,a) - (r + \gamma \mathbb{E}_{a'\sim\pi_\phi(s,\cdot)}[Q_{\overline{\theta}}(s',a') - \alpha \log \pi(s',a')])\Big)^2]$$

where, the target $\overline{\theta}$ is an exponentially moving average of past soft Q-function weights. The gradient of $L_Q(\theta)$ estimated using a single sample (s,a,r,s') and $a' \sim \pi_\phi(s',\cdot)$ as

$$\hat{\nabla}_\theta L_Q(\theta) = \nabla_\theta Q_\theta(s,a)(Q_\theta(s,a) - (r + \gamma\mathbb{E}))$$

11.4.2 Policy update

Policy parameter update: SAC updates policy using the loss function

$$L_\pi(\phi) = \mathbb{E}_{s\sim\mathcal{D}}\left[D_{\mathrm{KL}}(\pi_\phi(s,\cdot),\ \frac{1}{Z_\theta(s)}\exp(Q_\theta(s,\cdot)/\alpha))\right]$$

This can be rewritten as

$$L_\pi(\phi) = \mathbb{E}_{s\sim\mathcal{D}}\mathbb{E}_{a\sim\pi_\phi(s,\cdot)}[\alpha \log \pi_\phi(s,a) - Q_\theta(s,a)]$$

Use $a \sim \mathcal{N}(\mu_\phi(s), I) \rightarrow a = \mu_\phi(s) + \epsilon = f_\phi(\epsilon; s)$, where ϵ is an input noise vector samples from a fixed distribution such as Gaussian. Then, we can rewrite the above loss function as

$$L_\pi(\phi) = \mathbb{E}_{s\sim\mathcal{D}}\mathbb{E}_{\epsilon\sim\mathcal{N}}[\log \pi_\phi(s, f_\phi(\epsilon; s)) - Q_\theta(s, f_\phi(\epsilon; s))]$$

The gradient can then be computed easily.

11.5 Twin Delayed Deep Deterministic Policy Gradient (TD3) Algorithm

11.5.1 Deterministic Policy Gradient

Recall the (stochastic) policy gradient theorem

$$\nabla V_{\pi_\theta} = \frac{1}{(1-\gamma)} \mathbb{E}_{(s,a)\sim\rho_{\pi_\theta}(\cdot,\cdot)}[\nabla_\theta \log \pi_\theta(s,a) Q_{\pi_\theta}(s,a)]$$

Theorem 11.5.1. Deterministic policy gradient theorem.

Let π_θ be a deterministic policy. Then

$$\nabla_\theta V_{\pi_\theta} = \frac{1}{(1-\gamma)} \mathbb{E}_{s\sim\rho_{\pi_\theta}(\cdot)}[\nabla_\theta \pi_\theta(s) \nabla_a Q_{\pi_\theta}(s,a)|_{a=\pi_\theta(s)}]$$

11.5.2 Q-value Estimation

In deep Q-learning, the Q-function parameter

$$s \sim \rho_{\pi_\theta}(\cdot)[\nabla_\theta \pi_\theta(s)]$$